

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут телекомунікацій, радіоелектроніки та електронної техніки



Лабораторна робота
З дисципліни
“Об'єктно-орієнтоване програмування, частина 2”

Виконав: ст.гр IX-22

Горячий Ростислав

Прийняв:

Гордійчук-Бублівська О.В

Львів – 2026

Мета:

Ознайомитися з принципами асинхронного програмування в Python за допомогою модуля `asyncio`, навчитися працювати з асинхронними запитами до бази даних через `SQLAlchemy`, реалізувати систему збору та обробки даних про мережеві вузли.

Створити базу даних та таблицю `nodes`, яка зберігатиме інформацію про мережеві вузли (ID, IP-адресу, статус).

```
et.py > setup_database
1  import sqlite3
2
3  def setup_database():
4
5      conn = sqlite3.connect('network_inventory.db')
6      cursor = conn.cursor()
7      cursor.execute('''
8          CREATE TABLE IF NOT EXISTS nodes (
9              id INTEGER PRIMARY KEY AUTOINCREMENT,
10             ip_address TEXT NOT NULL UNIQUE,
11             status TEXT DEFAULT 'unknown',
12             last_seen TIMESTAMP DEFAULT CURRENT_TIMESTAMP
13         )
14     ''')
15     nodes_to_add = [
16         ('192.168.1.1', 'online'),
17         ('192.168.1.5', 'offline'),
18         ('10.0.0.12', 'maintenance')
19     ]
20
21     try:
22         cursor.executemany('''
23             INSERT OR IGNORE INTO nodes (ip_address, status)
24             VALUES (?, ?)
25         ''', nodes_to_add)
26
27         conn.commit()
28         print("Базу даних та таблицю 'nodes' успішно створено!")
29     except sqlite3.Error as e:
30         print(f"Виникла помилка: {e}")
31     finally:
32         conn.close()
33
34     if __name__ == "__main__":
35         setup_database()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Базу даних та таблицю 'nodes' успішно створено!
PS C:\Users\w1olf\Downloads\dwd>

Реалізувати асинхронну функцію для отримання списку вузлів із бази.

```
et.py > fetch_nodes
1 import asyncio
2 import aiosqlite
3
4 async def fetch_nodes():
5     db_path = 'network_inventory.db'
6     async with aiosqlite.connect(db_path) as db:
7
8         db.row_factory = aiosqlite.Row
9
10        async with db.execute("SELECT id, ip_address, status FROM nodes") as cursor:
11            rows = await cursor.fetchall()
12
13            print(f"{'ID':<5} | {'IP Address':<15} | {'Status':<12}")
14            print("-" * 35)
15
16            for row in rows:
17
18                print(f"{row['id']:<5} | {row['ip_address']:<15} | {row['status']:<12}")
19
20 if __name__ == "__main__":
21     asyncio.run(fetch_nodes())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\w1olf\Downloads\dwd> & C:/Users/w1olf/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/w1olf/Downloads/dwd/et.py
ID | IP Address | Status
-----
1 | 192.168.1.1 | online
2 | 192.168.1.5 | offline
3 | 10.0.0.12 | maintenance
PS C:\Users\w1olf\Downloads\dwd>
```

Реалізувати асинхронну систему збору статусів вузлів (імітація мережевих запитів).

```
1 import asyncio
2 import aiosqlite
3 import random
4
5 async def check_node_status(ip):
6     print(f"[...] Перевірка вузла {ip}...")
7     await asyncio.sleep(random.uniform(0.5, 2.0))
8     is_online = random.choice([True, True, False])
9     status = "online" if is_online else "offline"
10
11    print(f"[OK] Вузол {ip} статус: {status}")
12    return ip, status
13
14 async def update_network_statuses():
15     db_path = 'network_inventory.db'
16
17     async with aiosqlite.connect(db_path) as db:
18         async with db.execute("SELECT ip_address FROM nodes") as cursor:
19             rows = await cursor.fetchall()
20             ips = [row[0] for row in rows]
21
22         if not ips:
23             print("База даних порожня. Спочатку додайте вузли.")
24             return
25
26         print(f"Починаємо опитування {len(ips)} вузлів...\n")
27         tasks = [check_node_status(ip) for ip in ips]
28         results = await asyncio.gather(*tasks)
29
30         await db.executemany(
31             "UPDATE nodes SET status = ?, last_seen = CURRENT_TIMESTAMP WHERE ip_address = ?",
32             [(status, ip) for ip, status in results]
33         )
34         await db.commit()
35         print(f"\n[УСПІХ] Всі статуси в базі даних оновлено.")
36
37 if __name__ == "__main__":
38     try:
39         asyncio.run(update_network_statuses())
40     except KeyboardInterrupt:
41         pass
```

```
[...] Перевірка вузла 10.0.0.12...
[...] Перевірка вузла 192.168.1.1...
[...] Перевірка вузла 192.168.1.5...
[OK] Вузол 192.168.1.5 статус: offline
[OK] Вузол 192.168.1.1 статус: online
[OK] Вузол 10.0.0.12 статус: offline
Traceback (most recent call last):
```

Продемонструвати та пояснити результати виконання програми (створення таблиці, додавання 10-15 вузлів, моніторинг з оновленням статусу, виведення списку вузлів до та після оновлення і т.д.).

```
1  import asyncio
2  import aiosqlite
3  import random
4  from datetime import datetime
5
6  DB_NAME = 'network_monitor.db'
7
8  async def init_db():
9      async with aiosqlite.connect(DB_NAME) as db:
10         await db.execute("DROP TABLE IF EXISTS nodes")
11         await db.execute('''
12             CREATE TABLE nodes (
13                 id INTEGER PRIMARY KEY AUTOINCREMENT,
14                 ip_address TEXT,
15                 status TEXT,
16                 last_seen TIMESTAMP
17             )
18         ''')
19
20         nodes = [(f"192.168.1.{i}", "unknown", None) for i in range(1, 16)]
21         await db.executemany(
22             "INSERT INTO nodes (ip_address, status, last_seen) VALUES (?, ?, ?)",
23             nodes
24         )
25         await db.commit()
26         print(f"[!] Базу створено. Додано 15 вузлів.\n")
27
28  async def ping_node(ip):
29      await asyncio.sleep(random.uniform(0.3, 1.5))
30      status = random.choice(["online", "online", "online", "offline"])
31      return ip, status
32
33  async def display_nodes(label):
34      print(f"\n--- {label} ---")
35      print(f"{'ID':<4} | {'IP Address':<15} | {'Status':<10} | {'Last Seen'}")
36      print("-" * 55)
37      async with aiosqlite.connect(DB_NAME) as db:
38         async with db.execute("SELECT * FROM nodes") as cursor:
39             async for row in cursor:
40                 last_seen = row[3] if row[3] else "Never"
41                 print(f"{row[0]:<4} | {row[1]:<15} | {row[2]:<10} | {last_seen}")
42
43  async def run_monitoring():
44      await init_db()
45      await display_nodes("СТАН ДО ОНОВЛЕННЯ")
```

```

46     print("\n[ПОШУК] Починаємо асинхронне опитування вузлів...")
47     async with aiosqlite.connect(DB_NAME) as db:
48         async with db.execute("SELECT ip_address FROM nodes") as cursor:
49             ips = [row[0] for row in await cursor.fetchall()]
50             results = await asyncio.gather(*(ping_node(ip) for ip in ips))
51             now = datetime.now().strftime("%H:%M:%S")
52             update_data = [(status, now, ip) for ip, status in results]
53
54             await db.executemany(
55                 "UPDATE nodes SET status = ?, last_seen = ? WHERE ip_address = ?",
56                 update_data
57             )
58             await db.commit()
59             await display_nodes("СТАН ПІСЛЯ МОНИТОРИНГУ")
60
61 if __name__ == "__main__":
62     asyncio.run(run_monitoring())

```

--- СТАН ДО ОНОВЛЕННЯ ---			
ID	IP Address	Status	Last Seen
1	192.168.1.1	unknown	Never
2	192.168.1.2	unknown	Never
3	192.168.1.3	unknown	Never
4	192.168.1.4	unknown	Never
5	192.168.1.5	unknown	Never
6	192.168.1.6	unknown	Never
7	192.168.1.7	unknown	Never
8	192.168.1.8	unknown	Never
9	192.168.1.9	unknown	Never
10	192.168.1.10	unknown	Never
11	192.168.1.11	unknown	Never
12	192.168.1.12	unknown	Never
13	192.168.1.13	unknown	Never
14	192.168.1.14	unknown	Never
15	192.168.1.15	unknown	Never

Висновок:

Під час виконання лабораторної роботи я ознайомився з принципами асинхронного програмування в середовищі Python, використовуючи модуль **asyncio**. Практична частина була зосереджена на реалізації асинхронної взаємодії з базами даних шляхом впровадження ORM-інструментарію **SQLAlchemy**